CS781 Capstone Project: In-House Legal Intelligence System

Manish Kumar
251330604
{manishkmr25}@iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)

Abstract

A law firm cannot send confidential case files to external language-model services, so every component of this system runs on in-house hardware and starts from models we trained ourselves. We pre-trained four 82–89M-parameter legal language models (decoder-only, encoder–decoder, prefix-LM, and an efficient decoder with grouped-query and sliding-window attention) on a cleaned 1B-token sample of the official Indian legal corpus, after an audit found validation leakage, duplicates and OCR noise in the starter data. Each of the six downstream tasks (entity recognition, rhetorical roles, judgment prediction, statute identification, summarization and prior-case retrieval) fine-tunes these backbones with six to nine adaptation techniques, including LoRMA and knowledge distillation. Long documents are handled by chunking, condensation and, for retrieval, citation-context windows. Cleaning the corpus lowered A4 validation perplexity from 13.3 to 11.3 at an equal token budget, reading the last rather than the first chunks raised judgment-prediction macro-F1 from 0.754 to 0.785, and retrieving per citation raised micro-F1@7 from 0.32 to 0.43.

1 Competition Result

Codabench username: _____ **Roll number:** 251330604

Best valid score for each phase and the leaderboard rank at the time of report freeze.

Table 1: Official phase scores and ranks. Fill every cell. State the date on which ranks were recorded.

Phase	Task	Primary metric (value)	Rank	Valid submissions
1	Legal LM Pretraining	BERTScore F1: 0.8286	95	6
2	Legal NER	strict macro-F1: 0.4970	80	6
3	Rhetorical role prediction	macro-F1: 0.5750	15	18
4	Court judgment prediction	macro-F1: 0.8121	13	18
5	Legal statute identification	macro-F1: 0.4624	6	16
6	Abstractive summarization	BERTScore F1: 0.8558	4	16
7	Prior case retrieval	micro-F1@7: 0.4950	_____	16

Total valid Codabench submissions across all phases: 96 **Date ranks were recorded:** 6 November 2026

Every phase has at least six distinct finished submissions, above the required four; the six best of each phase and their methods are listed in Appendix A.

Table 2: Tasks (inputs, outputs and primary metrics as scored on Codabench).

Ph.	Task	Input → output	Primary metric
1	Legal LM	512-word legal prompt → 128-word continuation	BERTScore F1
2	L-NER	judgment text → character spans with one of 12 entity types	strict macro-F1
3	Rhetorical roles	sentences of a judgment → one of 13 roles each (Corporate Law, IT)	macro-F1
4	Judgment prediction	judgment (outcome removed) → allowed (1) / dismissed (0)	macro-F1
5	Statute identification	case text (statute names masked) → set of statute ids (of 100)	macro-F1
6	Summarization	judgment → abstractive summary	BERTScore F1
7	Prior case retrieval	query judgment + candidate pool → 7 ranked case ids	micro-F1@7

2 Introduction and Problem Description

A law firm that handles confidential matters needs language technology it can run on its own hardware: it cannot send case files to commercial APIs, and it cannot fine-tune an outside foundation model whose training data and licence it does not control. The capstone therefore asks for a legal NLP stack built from a compact language model pre-trained in house (50–300M parameters, no pretrained weights, no external corpora, official data from `Exploration-Lab/CS-781-Capstone` only), which is then adapted to six downstream tasks drawn from the IL-TUR benchmark [1]. Table 2 states each task.

Beyond the parameter cap and the data lock, three properties shaped every design decision. First, most documents are long (median 2,469 tokens in Phase 4, 3,123 words in Phase 6 and 4,508 words in Phase 7) while the Phase-1 models have a 1,024-token context. Second, the metrics are strict: NER counts only exact character spans, Phases 2–5 average F1 over classes so rare labels count fully, and summaries are scored against a single reference. Third, compute was one GPU shared with other users.

We investigated whether cleaning the pre-training corpus matters more than architecture, how the four backbones transfer to each task, which adaptation techniques keep full-fine-tuning quality with few trainable parameters, how to represent documents four to eight times longer than the context, and how decoding and thresholding choices move the metrics. We built and submitted every phase end to end; we did not pipeline the phases, train task-specific tokenizers, or run a human legal evaluation.

3 Data Analysis and Preprocessing

3.1 Sources and official splits

Only the official dataset and its phase configurations were used. Table 3 summarises the splits as used; integrity problems found in the provided files are listed below it.

Integrity findings. Each of the following changed the protocol. The Phase-1 starter split contained validation documents copied into training, so we rebuilt a validation set that shares no text with either corpus. In Phase 4, `test.json` carries labels and all 1,517 test documents occur, labelled, in `dev.json`; we never read those labels and selected models only on the 1,988 development documents not in the test set. In Phase 5, development texts that also occur in the test file (96) or in training (118) were removed, and 435 duplicate training texts merged. In Phase 7, every query occurs in its own candidate pool (never as its own gold case), and a pool is essentially the union of its queries' gold cases: 99.9% (train) and 100% (dev) of pool cases that are not themselves queries are gold for some query of the split.

Table 3: Data per phase as used (after de-duplication and leak removal). Lengths are medians.

Ph.	Split sizes	Statistics that informed the design
1	1.006B train tokens (cleaned), 4.10M val tokens	3.48M documents read; 106,717 exact and 204,246 near duplicates removed; 90,856 documents with OCR word stutter repaired; 764 validation documents dropped as leaks
2	70 train docs (10 held out), 35 test	8,393 entities in 12 types; median 4,488 words per document
3	90 labelled docs (45 CL, 45 IT), 10 test	labels persist: next sentence has the same role with probability 0.94 (CL) / 0.75 (IT); PrecedentOverruled has 3 training sentences
4	31,677 train, 1,988 clean val, 1,517 test	58.6% of training labels are 0, validation and test balanced; median 2,469 tokens (train), 3,771 (val)
5	42,315 train, 10,024 val, 13,019 test	100 statutes, 3.71 per document; frequency 270–11,577 documents; statute names masked as `<SECTION>`/`<ACT>`
6	6,703 train, 300 val (ours), 100 test	documents 3,123 words, summaries 640 words (compression 4.6); 75% of references exceed the ≈400 words that BERTScore compares
7	827 / 118 / 237 queries; pools 4,320 / 1,023 / 1,727	6.4–8.2 gold cases per query (range 0–92); citations masked as `<CITATION>`; a perfect 7-item list scores only 0.618 on dev

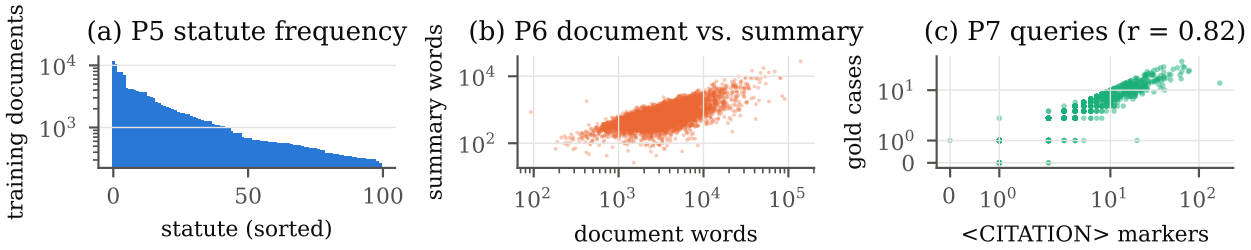


Figure 1: (a) Phase-5 statutes per training document count, sorted (log scale). (b) Phase-6 document length against summary length. (c) Phase-7 training queries: number of `<CITATION>` markers against number of gold cases.

3.2 Preprocessing decisions

All models share one byte-level BPE tokenizer [2] (32,000 tokens, trained on legal text), so Devanagari names, section symbols and OCR debris never become unknown tokens. Phase-1 cleaning normalised characters and markup, repaired word stutter, filtered short or non-textual documents and removed exact and MinHash near-duplicates. For NER, every span is mapped from characters to BPE tokens through the tokenizer's offset table; runs of punctuation are split before encoding because they otherwise merge with entity boundaries (the span-to-tag-to-span round trip recovers 99.45% of gold spans instead of 98.66%). Phase 3 uses the provided sentence lists as they are. Long documents are handled per task: eight 512-token chunks (last eight in Phase 4, evenly spread in Phase 5), a condensed source of 1,280–8,192 tokens in Phase 6 (the opening of the case plus the sentences closest to the document's TF-IDF centroid), and citation windows of ±60 words plus 512-token candidate chunks in Phase 7.

4 Approach and Model Evolution

4.1 Phase-1 foundation model

Four architectures share tokenizer, data, schedule and evaluation, so only the architecture differs (Table 4); parameter counts are summed over the instantiated models (tied embeddings). All are trained with next-token cross-entropy on 1,024-token windows; A2 learns to continue a random-length prefix and A3 reads a random prefix bidirectionally. The first runs (R0) used the starter sample of about 200M tokens. After the audit described above, we retrained on the cleaned corpus

Table 4: The four Phase-1 architectures (vocabulary 32k, context 1,024).

ID	Design	Params
A1	Decoder-only [4]; learned positions, pre-LayerNorm, GELU; 8 layers, $d = 768$, 12 heads, $d_{\text{ff}} = 3072$	82.1M
A2	Encoder–decoder [5], 6+6 layers, sinusoidal positions, $d = 640$, 10 heads	89.4M
A3	Prefix-LM: bidirectional prompt, causal continuation, ALiBi [6]; 8 layers, $d = 768$	81.3M
A4	RoPE [7], RMSNorm, SwiGLU, grouped-query attention [8] (12 q / 4 kv heads), 256-token sliding window [9] with every third layer global; 10 layers, $d = 768$	87.5M

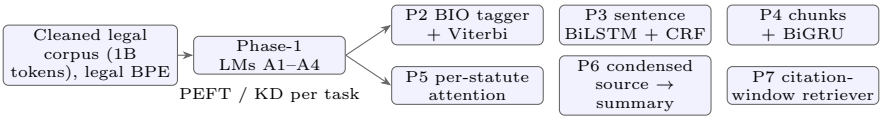


Figure 2: One pre-trained backbone family, six separately fine-tuned descendants.

at the same budget (R1) and continued to 1B tokens (R2). The submission decoder samples with temperature 0.8, top-k 50, top-p 0.95 and repetition penalty 1.1 over the last 256 tokens [3], and forbids end-of-text before 128 words.

4.2 Transfer protocol (Phases 2–7)

Every downstream model starts from a Phase-1 checkpoint (cleaned R2 from Phase 3 on; Phase 2 also used R0) and is fine-tuned on task data only (Figure 2). A shared wrapper exposes the backbones as encoders: A1 and A4 keep their causal mask (better than bidirectional in Phase 3), A3 reads a bidirectional prefix, and A2 contributes decoder states because its Phase-1 encoder output is nearly collapsed. Adaptation techniques were implemented once and reused: full and partial fine-tuning, gradual unfreezing [10], BitFit [11], LoRA [12], LoRMA [13], QLoRA with NF4 weights [14], AdaLoRA [15], Houlsby adapters [16], IA^3 [17], prefix [18] and prompt tuning [19], and knowledge distillation [20] into students that keep a subset of the teacher's blocks. Each phase used at least six of them across all four architectures.

4.3 Phase 2 — Legal NER

NER is token classification followed by exact offset reconstruction: gold spans become BIO tags over BPE tokens, each backbone tags overlapping 512-token windows (stride 128, each token keeps the window with the most context), and a Viterbi decoder that only allows valid BIO transitions turns tag scores back into character spans. Six experiments covered all four architectures.

4.4 Phase 3 — Rhetorical role prediction

Each document is encoded in windows of neighbouring sentences; every sentence is mean-pooled, a BiLSTM with relative-position and domain embeddings reads the document, and a sentence-level CRF [21] decodes the role sequence, matching the long label runs in Table 3. Corporate Law and IT were trained jointly with a domain embedding, under five-fold document-level cross-validation over the 90 labelled documents. A logit-adjusted loss gives rare roles more weight.

4.5 Phase 4 — Court judgment prediction

Eight 512-token chunks are encoded, mean-pooled, read by a BiGRU and pooled by attention [22]. The loss is class-weighted and the decision threshold is tuned on validation, because training is 59% label 0 while validation and test are balanced; macro-F1, not accuracy, makes both outcomes count equally.

Table 5: Best Codabench submission per phase, its checkpoint(s) and local held-out score.

Ph.	Submission	Checkpoint(s)	Local	Codabench
1	A1 R0	`runs/A1_gpt/best1.pt` (starter corpus, 200M tokens)	0.829	0.8286
2	E2	A4 LoRMA on `runs/A4_efficient/best.pt`, BIO Viterbi	0.486	0.4970
3	T14	A1 LoRA, causal, 1,024-token windows, 5 fold models	0.501	0.5750
4	STACK_with_fusion	logistic-regression stacker over 26 runs (Appendix A)	0.823	0.8121
5	STACK11_full	MLP + LightGBM stackers, 28 Phase-1 + 6 lexical models	0.460	0.4624
6	S3 beam 4	A4 full fine-tuning, 4,096-token source, 4 beams	0.852	0.8558
7	STK4	LightGBM re-ranker + mean of 4 bi-encoders	0.491	0.4950

4.6 Phase 5 — Legal statute identification

The same chunk encoder feeds a BiGRU and one attention query per statute, so each statute reads the part of the judgment that discusses it, followed by 100 independent sigmoids. Binary cross-entropy uses positive weights $\sqrt{n_{\text{neg}}/n_{\text{pos}}}$ (capped at 30) for rare statutes, and a threshold is tuned per statute on validation. Later rounds distilled 6-block students from ensembles of earlier runs, and MLP and per-statute LightGBM [23] stackers were trained out-of-fold on validation over all runs' logits, optionally with TF-IDF models (a hybrid without pretrained weights).

4.7 Phase 6 — Abstractive summarization

The decoder-only models read [prompt, condensed source, "`Summary:`"] and are trained on the summary tokens; A2 uses fusion-in-decoder [24] over four 1,024-token chunks. A1's position table was linearly interpolated [25] (ALiBi and RoPE extrapolate) so that sources could grow from 1,280 to 8,192 tokens, each run starting from the previous shorter-source checkpoint. Decoding is greedy with repetition penalty 1.15 and no repeated 3-grams. The final step selects, per document, the candidate summary that agrees most with the other candidates (minimum-Bayes-risk selection [26] with ROUGE-1 + ROUGE-2 agreement). No preference optimisation or other post-training was used.

4.8 Phase 7 — Prior case retrieval

The text around each masked citation describes one cited case, so retrieval works per citation window. The lexical stage issues one BM25 [27] query per window, penalises candidates dated after the query, and normalises each candidate's scores across the split's queries and windows, because every pool case is cited by some query. The neural stage, a bi-encoder on a Phase-1 backbone, scores a window by its best-matching 512-token candidate chunk and is trained with multi-positive InfoNCE [28] and BM25 hard negatives. A LightGBM lambdarank model re-ranks the top 100 lexical candidates, and the mean score of 13 trained retrievers is added to its output.

4.9 Final submitted system

5 Experiments and Implementation Details

5.1 Compute and training protocol

All runs used one NVIDIA H200 shared with other users (two to four concurrent runs). Phase-1 R2 processed 803M tokens in 0.8–1.1 hours per architecture; downstream runs took 2 minutes to 3.5 hours. All used AdamW, warm-up and cosine decay, clipping at 1.0 and bf16, kept their best state

Table 6: Hyperparameters of the final model of each phase (P5–P7: the best single Phase-1 model). *R2, after 3,000 steps of R1.

Hyperparameter	P1 (A4)	P2 (A4 LoRMA)	P3 (A4 LoRMA)	P4 (A4 partial)	P5 (A4 KD)	P6 (A1 full)	P7 (A1 full)
Parameters (M)	87.5	87.5 (2.2 trained)	87.5	88.0 (25.7 tr.)	63.4	85.0	82.3
Context length	1,024	512 windows	512 windows	8×512	8×512	4,096 + 768	256 / 512
Peak LR	6×10^{-4}	10^{-3}	10^{-3}	10^{-4}	5×10^{-5}	5×10^{-5}	3×10^{-5}
Batch × accum.	8 × 8 seq.	16 windows	4 chunks	2 × 8 docs	4 × 4 docs	1 × 16	8 queries
Epochs / steps	12,260 steps*	≤15 (patience 5)	20 (patience 6)	2	8	3 (+6 at 2,560)	10 (best 5)
Dropout	0	0.1	0.1	0.1	0.1	0.1	0
Decoding / threshold	T 0.8, top-p 0.95	BIO Viterbi	CRF Viterbi	0.39	per statute	greedy, pen. 1.15	top 7

Table 7: One-factor comparisons on the local held-out split of each phase. †Three changes at once: causal mask, 20 epochs, logit adjustment $\tau = 0.5$. OOF = out-of-fold, CF = cross-fitted.

Ph.	Factor changed (everything else fixed)	Model, metric	Before	After
1	Corpus cleaning, same 200M tokens	A4, fair-val perplexity	13.32	11.25
1	Token budget 200M → 1B	A4, fair-val perplexity	11.25	8.26
2	Argmax → BIO Viterbi decoding	A4 LoRMA, macro-F1	0.373	0.486
2	Original → cleaned 1B backbone	A4 LoRMA, macro-F1 (3 splits)	0.488	0.467
3	Sentence alone → context window	A4 LoRMA, OOF macro-F1	0.361	0.455
3	Tuning round†	A4 LoRMA, OOF macro-F1	0.455	0.500
4	First 8 → last 8 chunks	A4 partial, macro-F1	0.754	0.785
5	3 → 5 epochs	A4 partial, CF macro-F1	0.370	0.381
5	Spread → statute-placeholder chunks	A4 partial, CF macro-F1	0.370	0.371
5	KD teacher 5 → 8 models	6-block A4 student, CF macro-F1	0.404	0.402
6	Repetition penalty + no repeated 3-grams	9 first-round runs, BERTScore	+0.003 to +0.008	
6	Source 1,280 → 2,560 tokens	A1 full, BERTScore F1	0.8505	0.8520
6	Spread → salient condensation	A1 LoRA, BERTScore F1	0.8452	0.8493
7	Whole query → per-citation BM25	no learning, micro-F1@7	0.324	0.432
7	+ case-wise pool normalisation	BM25 + date penalty, micro-F1@7	0.453	0.469
7	No teacher → distillation from E2	4-block A1, neural micro-F1@7	0.354	0.369

and were resumable. Selection used validation loss (P1, P6) or the official metric on the held-out split of subsection 5.2 (P2–P5, P7).

5.2 Local evaluation

Held-out splits use public data only: a leak-free Phase-1 validation set, 10 of the 70 NER training documents (confirmed on three splits), five-fold document-level cross-validation (P3), development documents not in the test set (P4), de-duplicated development data with cross-fitted thresholds (P5), 300 drawn training documents (P6), and the official dev split (P7). A script checked every submission's schema before zipping (no missing or extra ids, one label per sentence, unique statute ids, non-empty summaries, 7 distinct pool ids per query that never include the query). Local and Codabench scores can differ because some splits are small and the local BERTScore uses roberta-large, not the organisers' "legal BERTScore".

5.3 Ablations and controlled comparisons

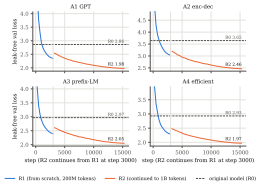
Table 7 changes one factor at a time; each row compares two runs that are otherwise identical.

6 Results

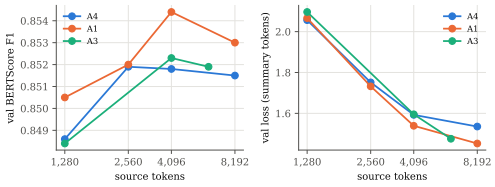
Table 8 lists the main variants with their local and Codabench scores; Figure 3 shows learning curves. Per class, two Phase-3 roles have almost no training data (PrecedentOverruled: 3 sentences; Dissent: 2 documents), and in Phase 5 the mean F1 of the 20 rarest statutes is 0.32 against 0.56 for the 20 most frequent (best single model).

Table 8: Main variants: local held-out score next to the Codabench test score (– = not uploaded). Secondary (local): P1 ROUGE-L/BLEU-4, P2 and P4 precision/recall, P5 micro-F1, P6 ROUGE-1/ROUGE-L, P7 precision/recall.

Ph.	Variant	Detail	Local	Secondary	Codabench
1	A1 R0	starter corpus, 200M tokens	0.829	0.192 / 0.058	0.8286
1	A4 R2	cleaned, 1B tokens	0.836	0.213 / 0.093	–
2	E2 A4 LoRMA	2.2M trained parameters	0.486	0.517 / 0.478	0.4970
2	E6 KD student	4-block A3, 52.9M parameters	0.344	0.372 / 0.344	0.3799
3	T14 A1 LoRA	tuned, 1,024-token windows	0.501	–	0.5750
3	C7 ensemble	8 taggers, out-of-fold	0.521	–	0.5707
4	E6 A4 partial	best single model	0.785	0.787 / 0.786	0.7633
4	ENS_top5	mean of 5 runs	0.797	0.800 / 0.798	0.7930
4	STACK_with_fusion	stacker, nested estimate	0.823	–	0.8121
5	STACK6	stacker, Phase-1 runs only	0.423	0.520	0.4231
5	STACK11	hybrid stacker (+ TF-IDF)	0.460	0.558	0.4624
6	F1 A4 full	1,280-token source, first decode	0.843	0.477 / 0.251	0.8483
6	S7 A1 full	4,096-token source, greedy	0.854	0.513 / 0.273	0.8508
6	S3 A4 full	4,096-token source, beam 4 (100 docs)	0.852	0.525 / 0.275	0.8558
7	STK0	lexical re-ranker only	0.477	0.519 / 0.441	0.4725
7	STK4	re-ranker + 4 bi-encoders	0.491	0.535 / 0.454	0.4950



(a) Phase 1: validation loss against training tokens.



(b) Phase 6: BERTScore and loss against source length.

Figure 3: Learning curves and the source-length experiment.

To be written by the author — which system won, and why. Which variant won in each phase, with a causal explanation supported by the comparisons in Table 7 (not "it trained longer").

7 Error Analysis and Discussion

7.1 Phase-wise failure modes

From the phase reports' error analyses: in Phase 2, 37.9% of gold entities are not predicted at all, 8.0% have a wrong boundary and 5.6% a wrong type; the most frequent type confusion is appellant → respondent (35 cases). In Phase 3 the largest confusions are Ratio ↔ PrecedentRelied (548 and 539 sentences) and Fact → Ratio (499). In Phase 4 the best single model makes 176 false positives and 250 false negatives on 1,988 validation judgments. In Phase 5 the most frequent error pair is a missed statute replaced by the most frequent statute (id 5) in the same document. In Phase 6 per-document BERTScore correlates −0.40 with log document length. In Phase 7 eight of 118 dev queries receive no gold case, and the precision of queries with one to three gold cases is capped by the fixed list length (P 0.21, R 0.81).

To be written by the author — failure modes and mechanisms. For each phase, examples, counts and a hypothesised mechanism.

7.2 What the metrics do not measure

To be written by the author — metrics vs. legal correctness. Discuss ROUGE/BERTScore against legal faithfulness, strict span-F1, and the fixed 7-item list in Phase 7, and what that means for a real deployment.

7.3 Limitations and what you would do next

To be written by the author — limitations and next experiments. Compute, context length, no external corpora, no human legal evaluation, single seeds on small validation splits, the hybrid (non-Phase-1) inputs in Phases 5 and 7; concrete next experiments.

7.4 Surprising findings

To be written by the author — surprising findings. For example: the cleaned backbone helped every phase except NER; pool structure and citation masking in Phase 7.

8 Conclusion

To be written by the author — conclusion. The system as a confidential on-prem stack, the two or three design choices that moved the metrics and those that did not, the main lesson, and what a sceptical partner should still not trust.

LLM Usage Declaration

Completing this section is mandatory. Inconsistent answers relative to the code and the prose will be treated as an integrity issue.

Q1. Which Large Language Model(s), if any, did you use while completing the competition or writing this report? Give model name and vendor (e.g., none / ChatGPT-4.x / Gemini x / Claude x / Grok / Copilot / other).

Answer: _____

Q2. For each row, mark Yes or No and state the purpose in one short phrase. "Yes" is acceptable only for the uses permitted by the capstone document.

Part	Used LLM?	Purpose of using the LLM
Understanding concepts / papers	Yes/No	
Ideation / brainstorming at a high level	Yes/No	
Debugging small ancillary snippets	Yes/No	
Plotting / visualization code	Yes/No	
Grammar / writing style of the report	Yes/No	
Report structure / section wording	Yes/No	
Designing the model architecture	Yes/No	
Hyperparameter search / tuning	Yes/No	
Analysis of results / conclusions	Yes/No	
Writing the core training / inference code	Yes/No	
Other (specify)	Yes/No	

Q3. Did you refer to sources other than lecture slides and an LLM? If yes, list them (papers, blogs, GitHub repositories, textbooks, official starter notebook, evaluation scripts). These must also appear in the bibliography when they influenced the method.

Answer:

Q4. Apart from any permitted LLM use, what was *your* contribution? In 8–12 sentences describe your own understanding, architecture decisions, implementation, experiments, and analysis. Name at least two things you tried that failed and what you concluded from them.

GitHub repository: https://github.com/...

References

[1] A. Joshi, S. Paul, A. Sharma, P. Goyal, S. Ghosh, and A. Modi, "IL-TUR: Benchmark for Indian legal text understanding and reasoning," *arXiv preprint arXiv:2407.05399*, 2024.

[2] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," in *Proceedings of ACL*, 2016.

[3] N. S. Keskar, B. McCann, L. R. Varshney, C. Xiong, and R. Socher, "CTRL: A conditional transformer language model for controllable generation," *arXiv preprint arXiv:1909.05858*, 2019.

[4] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI technical report*, 2019.

[5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in Neural Information Processing Systems*, 2017.

[6] O. Press, N. A. Smith, and M. Lewis, "Train short, test long: Attention with linear biases enables input length extrapolation," in *International Conference on Learning Representations*, 2022.

[7] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *arXiv preprint arXiv:2104.09864*, 2021.

[8] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proceedings of EMNLP*, 2023.

[9] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The long-document transformer," *arXiv preprint arXiv:2004.05150*, 2020.

[10] J. Howard and S. Ruder, "Universal language model fine-tuning for text classification," in *Proceedings of ACL*, 2018.

[11] E. Ben Zaken, Y. Goldberg, and S. Ravfogel, "BitFit: Simple parameter-efficient fine-tuning for transformer-based masked language-models," in *Proceedings of ACL*, 2022.

[12] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations*, 2022.

[13] H. Bihany, S. Patel, and A. Modi, "LoRMA: Low-rank multiplicative adaptation for LLMs," in *Findings of ACL*, 2025.

[14] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems*, 2023.

[15] Q. Zhang, M. Chen, A. Bukharin, P. He, Y. Cheng, W. Chen, and T. Zhao, "AdaLoRA: Adaptive budget allocation for parameter-efficient fine-tuning," in *International Conference on Learning Representations*, 2023.

[16] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. De Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, "Parameter-efficient transfer learning for NLP," in *International Conference on Machine Learning*, 2019.

[17] H. Liu, D. Tam, M. Muqeeth, J. Mohta, T. Huang, M. Bansal, and C. Raffel, "Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning," in *Advances in Neural Information Processing Systems*, 2022.

[18] X. L. Li and P. Liang, "Prefix-tuning: Optimizing continuous prompts for generation," in *Proceedings of ACL*, 2021.

[19] B. Lester, R. Al-Rfou, and N. Constant, "The power of scale for parameter-efficient prompt tuning," in *Proceedings of EMNLP*, 2021.

[20] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.

[21] J. Lafferty, A. McCallum, and F. Pereira, "Conditional random fields: Probabilistic models for segmenting and labeling sequence data," in *International Conference on Machine Learning*, 2001.

[22] Z. Yang, D. Yang, C. Dyer, X. He, A. Smola, and E. Hovy, "Hierarchical attention networks for document classification," in *Proceedings of NAACL*, 2016.

[23] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, 2017.

[24] G. Izacard and E. Grave, "Leveraging passage retrieval with generative models for open domain question answering," in *Proceedings of EACL*, 2021.

[25] S. Chen, S. Wong, L. Chen, and Y. Tian, "Extending context window of large language models via positional interpolation," *arXiv preprint arXiv:2306.15595*, 2023.

[26] S. Kumar and W. Byrne, "Minimum Bayes-risk decoding for statistical machine translation," in *Proceedings of HLT-NAACL*, 2004.

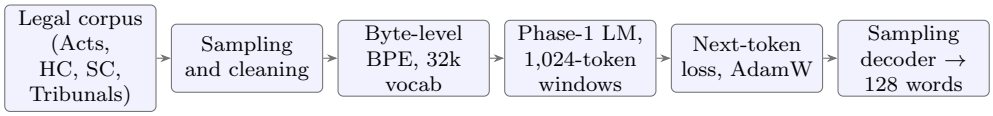
[27] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," in *Foundations and Trends in Information Retrieval*, 2009.

[28] A. van den Oord, Y. Li, and O. Vinyals, "Representation learning with contrastive predictive coding," *arXiv preprint arXiv:1807.03748*, 2018.

A Method details of the top-6 submissions

For every phase: the shared pipeline (diagram), its common settings, and the six best finished Codabench submissions with the settings that distinguish them. "Local" is the held-out score of the same prediction: P1 dev BERTScore F1 (200 leak-free prompts); P2 strict macro-F1 (10 held-out documents); P3 out-of-fold macro-F1 (5 folds); P4 clean-validation macro-F1 at the tuned threshold (stacks: nested split-half); P5 cross-fitted macro-F1; P6 BERTScore F1 (300 validation documents; beam 4: 100); P7 dev micro-F1@7. Notation: r = low-rank dimension, α = scaling, τ = logit-adjustment strength, KD = knowledge distillation with loss $\alpha\,\mathrm{CE} + (1-\alpha)T^2\,\mathrm{KL}$.

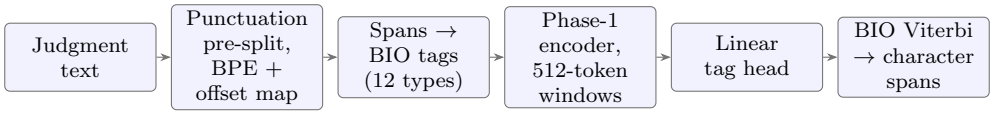
A.1 Phase 1 — Legal language model pre-training



Common: next-token cross-entropy; AdamW (6×10^{-4}, cosine, 200 warm-up steps); 65,536 tokens per step; best checkpoint by validation loss. **Decoding:** temperature 0.8, top-k 50, top-p 0.95, repetition penalty 1.1 over the last 256 tokens, end-of-text banned before 128 words. All six uploads are R0 models (starter sample, 200M tokens); the cleaned 1B-token models (R2) were trained after the Phase-1 deadline.

#	Submission	Codabench	Local	Method (key settings)
1	submission_2026-09-26_22-29-00	0.8286	0.8286	**A1 GPT**: decoder-only, learned positions, pre-LayerNorm, GELU; 8 layers, $d = 768$, 12 heads; 82.1M.
2	submission_2026-09-27_01-22-10	0.8285	0.8255	**A3 prefix-LM**: bidirectional prefix, causal continuation, ALiBi positions; 8 layers, $d = 768$; 81.3M.
3	submission_2026-09-27_01-29-15	0.8280	0.8269	**A4 efficient**: RoPE, RMSNorm, SwiGLU, grouped-query attention (12 q / 4 kv), 256-token sliding window, global layer every 3rd; 10 layers; 87.5M.
4	submission_2026-09-26_22-38-32	0.8184	0.8170	**A2 encoder–decoder**: 6+6 layers, sinusoidal positions, $d = 640$; trained on random prefix/continuation splits; 89.4M.
5	submission.zip (26 Sep)	0.8006	–	Early upload before the four-architecture runs; the file does not identify the model.
6	submission.zip (25 Sep)	0.8002	–	Early upload, as above.

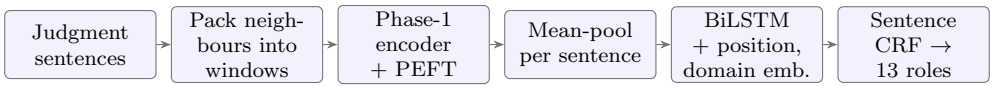
A.2 Phase 2 — Legal named-entity recognition



Common: windows of 512 tokens, stride 256 (training) / 128 (inference, each token keeps its best-context window); 16 windows per step; up to 15 epochs, patience 5; head learning rate 10^{-3}; selection by strict span macro-F1 on 10 held-out documents. Original (R0) Phase-1 backbones.

#	Submission	Codabench	Local	Method (key settings)
1	E2_a4_lorma	0.4970	0.4857	**A4 + LoRMA**: $W = (I + \frac{\alpha}{r}BA)W_0$, $r = 16$, $\alpha = 32$, on q, k, v, o, gate, up, proj; bidirectional attention; LR 10^{-3}; 2.2M trained.
2	E1_a3_full	0.4815	0.4267	**A3 full fine-tuning**: every weight; LR 10^{-4}; 81.3M trained.
3	E5_a4_partial	0.4753	0.4697	**A4 partial**: top 4 of 10 blocks + final norm trainable; bidirectional; LR 2×10^{-4}.
4	E3_a1_lora	0.4239	0.3564	**A1 + LoRA**: $r = 16$, $\alpha = 32$ on q, k, v, o, fc, proj; bidirectional; LR 10^{-3}; 1.8M trained.
5	E4_a2_adapter	0.4015	0.3892	**A2 + Houlsby adapters**: bottleneck 64 after attention and MLP; decoder states; LR 10^{-3}.
6	E6_kd_a3_small	0.3799	0.3443	**KD**: A3 student with blocks 1, 3, 5, 7 (52.9M), full fine-tuning; teacher E1; $\alpha = 0.5$, $T = 2$.

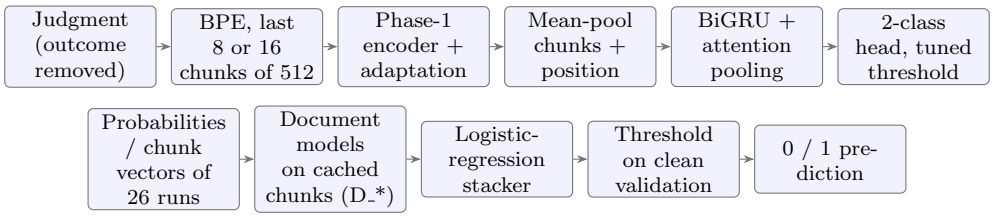
A.3 Phase 3 — Rhetorical role prediction



Common: cleaned 1B-token backbones; BiLSTM hidden 256; linear-chain CRF over sentences; logit-adjusted loss for rare roles; 5-fold document-level cross-validation over the 90 labelled documents (CL + IT, domain embedding); test emissions averaged over the 5 fold models. "Tuned" (T7–T22) = causal attention, 20 epochs (patience 6), $\tau = 0.5$. Ensembles average the CRF emissions of several runs; "nocal" = no per-class bias calibration.

#	Submission	Codabench	Local	Method (key settings)
1	T14_t8_win1024	0.5750	0.5007	**A1 + LoRA** ($r = 16$, $\alpha = 32$), tuned, **1,024-token windows** (matches the pre-training block length); 2.4M trained.
2	T15_t8_r32	0.5736	0.4926	As T8 with LoRA rank 32 ($\alpha = 64$); 4.1M trained.
3	C6_ens_8_nocal	0.5711	0.5200	**Ensemble of 8**: T14, T17 (BiLSTM 512), T18/T19 (seeds), T7 (A4 LoRMA), T13 (KD 6-block A4), T10 (A4 QLoRA), T9 (A3 rsLoRA).
4	T8_e2_causal_ep20_tau05	0.5709	0.4915	**A1 + LoRA**, tuned, 512-token windows.
5	C7_ens_fold_plus_all	0.5707	0.5211	**C7 ensemble** (as C6 with lexical cue flags in T20/T22): fold models plus their all-data retrains averaged.
6	C5_ens_a1x4_nocal	0.5696	0.5179	**Ensemble of 4 A1 LoRA taggers**: T14, T17, T18, T19.

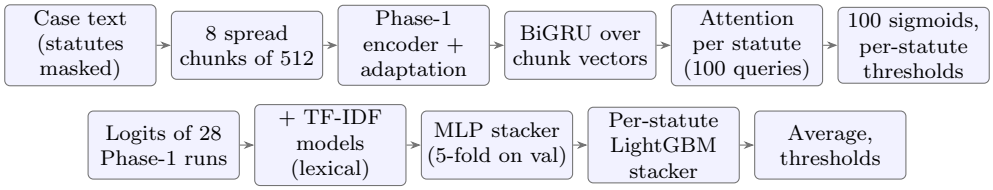
A.4 Phase 4 — Court judgment prediction



Common: class-weighted cross-entropy (train 59% label 0); 16 documents per step; 2 epochs; selection and threshold on the 1,988 development documents not in the test set. **Stackers** (rows 1–2): logistic regression ($\ell_2 = 0.01$) over every run's logit, fitted on clean validation. **D_* document models**: the last $\leq$48 chunks of each judgment embedded once by fine-tuned encoders (V1, V2, V3), concatenated, read by mean / attention / GRU / LSTM / 2-layer transformer layers.

#	Submission	Codabench	Local	Method (key settings)
1	STACK_with_fusion	0.8121	0.8225	**Stacker over 26 runs**: E1–E8, 3 chunk-selection ablations, V1–V6, 9 D_* document models (fusing 1, 3 or 5 encoders); threshold 0.52.
2	STACK_all	0.8048	0.8160	**Stacker over 17 end-to-end runs** (no document models); threshold 0.50.
3	ENS_greedy5	0.8009	0.8238	**Mean of 5**, greedy selection: V1 (A4 partial, 16 chunks), A4 on first chunks, E7 (A3 gradual unfreezing), E4 (A2 BitFit), V6 (6-block A4 KD from a 3-model ensemble).
4	ENS_8_views_kd	0.7963	0.8068	**Mean of 8**: E6, E7, E1, E3, E2, first+last and spread chunk views, E8 KD student; threshold 0.42.
5	ENS_greedy4	0.7955	0.8219	As ENS_greedy5 without V6; threshold 0.36.
6	ENS_top5	0.7930	0.7974	**Mean of 5 best first-round runs**: E6 (A4 partial), E7 (A3 gradual), E1 (A1 full), E3 (A3 adapters), E2 (A4 LoRMA+); threshold 0.39.

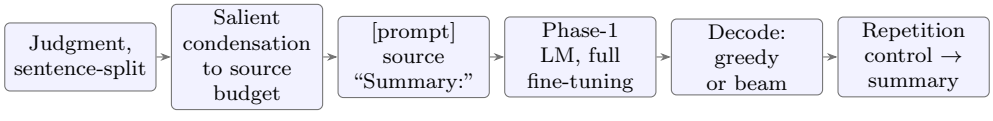
A.5 Phase 5 — Legal statute identification



Common: BCE with positive weights $\sqrt{n_{\text{neg}}/n_{\text{pos}}}$ (cap 30); 16 documents per step; per-statute thresholds fitted on validation (scored cross-fitted); KD students = A4 with 6 of 10 blocks distilled from averaged logits of 3–8 earlier runs. **Stackers:** trained 5-fold on the validation split (out-of-fold scores), fold models averaged for test. **MLP**: all runs' logits for all 100 statutes, hidden 512, dropout 0.3. **LightGBM**: per statute, each run's logit for it, the mean logit of all statutes, and model spread. Rows 1–5 are *hybrid* (TF-IDF logistic regression / SVM use no pretrained weights).

#	Submission	Codabench	Local	Method (key settings)
1	STACK11_full	0.4624	0.4603	MLP (2 seeds) + LightGBM (3 seeds) over 28 Phase-1 runs and 6 lexical models, incl. a TF-IDF model trained on train + validation folds.
2	STACK10_seedavg	0.4587	0.4602	As STACK9, averaged over independently seeded stackers.
3	STACK9_gbm_mlp	0.4579	0.4595	LightGBM stacker (32 inputs) averaged with the STACK8 MLP stacker.
4	STACK8_hybrid	0.4493	0.4473	MLP stacker, 27 Phase-1 runs + 4 lexical (word 1–2 / 1–3-gram, character TF-IDF, linear SVM).
5	STACK7_hybrid	0.4457	0.4436	MLP stacker, 27 Phase-1 runs + word TF-IDF logistic regression.
6	STACK6_all	0.4231	0.4233	MLP stacker over 27 Phase-1 runs only: all four architectures, 8 PEFT methods, 6 KD students.

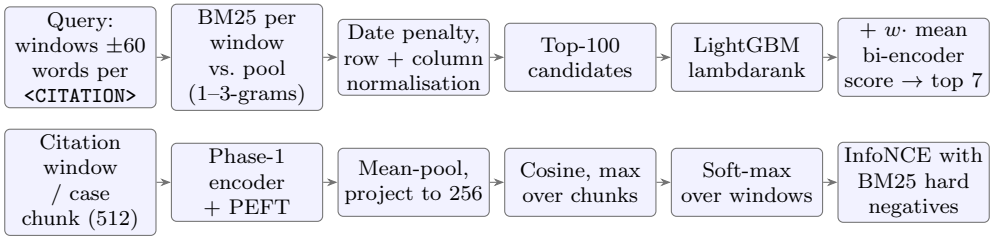
A.6 Phase 6 — Abstractive summarization



Common: loss on summary tokens only (up to 768); salient condensation = case opening (15% of the budget) + sentences nearest the document's TF-IDF centroid, in order; AdamW, 16 sequences per step; best state by validation loss. **Decoding:** 300–800 new tokens, repetition penalty 1.15, no repeated 3-grams. Long sources: A4 (RoPE) extrapolates; A1's learned positions are linearly interpolated; longer-source runs start from the shorter-source checkpoint.

#	Submission	Codabench	Local	Method (key settings)
1	S3_a4_full_src4096_beam4	0.8558	0.8522	**A4 full**, 4,096-token source, 5 epochs from Phase-1; **beam search, 4 beams**.
2	S4_a4_full_src8192	0.8539	0.8515	**A4 full**, 8,192-token source, 3 epochs continued from S3 (LR 5×10^{-5}); greedy.
3	S1_a4_full_src2560	0.8538	0.8519	**A4 full**, 2,560-token source, 6 epochs; greedy.
4	S8_a1_full_src8192	0.8520	0.8530	**A1 full**, 8,192-token source (interpolated positions), 2 epochs continued from S7 (4,096); greedy.
5	S3_a4_full_src4096_long	0.8520	0.8515	S3 re-decoded with 450–900 new tokens (longer summaries).
6	S3_a4_full_src4096	0.8520	0.8518	S3, greedy decoding.

A.7 Phase 7 — Prior case retrieval



Common: the query's own case is never returned; 7 distinct pool ids per query. **Re-ranker** (STK*): features = row and column z-scores and ranks of 7 lexical scores, date difference, case length, number of windows; trained on train (3 seeds), refit on train + dev for test. **Bi-encoder** (second diagram): 8 queries per step, 2 gold + 3 hard negatives each, temperature 0.05, best of 10 epochs on dev. **Fused** = z(neural) + λ z(BM25) − date penalty, weights picked on dev.

#	Submission	Codabench	Local	Method (key settings)
1	STK4_rc_blend4	0.4950	0.4914	Re-ranker with row + column features and first stage, **+ 0.5 × mean row + column score of 4 bi-encoders** (E1 A3 LoRMA, E2 A1 full, E3 A4 partial, E5 A3 IA^3).
2	STK2_lex_blend4	0.4773	0.4869	Re-ranker (row features only) + 0.3 × mean row score of the same 4 bi-encoders.
3	STK1_e2	0.4743	0.4725	Re-ranker with the out-of-fold E2 score (2-fold runs) as an extra feature.
4	STK0_lex	0.4725	0.4769	Lexical re-ranker only, no neural input.
5	K1_kd_a1_4l_fused	0.4649	0.4625	**KD**: A1 student with blocks 0, 2, 5, 7 (53.9M), batch-score KL to teacher E2 ($\alpha = 0.5$); fused with BM25.
6	E2_a1_full_fused	0.4643	0.4703	**A1 full fine-tuning** bi-encoder (bidirectional, LR 3×10^{-5}), fused with BM25.